SOF 103
C AND C++ PROGRAMMING

Lecture 1

Introductory Class

VISION AND MISSION

Vision

Xiamen University Malaysia aspires to become a university with a distinct global outlook, featuring first-class teaching and research, and embracing cultural diversity.

Mission

To nurture young talents with dignity and wisdom, turning them into fine citizens of the region who will contribute to the prosperity of the people and social progress of Malaysia, China and Southeast Asia.

LECTURER

Wong Chim Chwee

A1#363

chimchwee.wong@xmu.edu.my

Consultation Hours:

Monday 11am-12pm

Tuesday 11am-12pm, 3pm-5pm

ASSESSMENT CRITERIA

1. Continuous Assessments 50%
   - Assignment 20% (week 6)
   - Mid-term Exam (2 hrs) 30% (week 9)

2. Final Exam (2 hrs) 50%

MAIN COMMUNICATION MODE

1. **XMUM Moodle**

   https://l.xmu.edu.my

2. **Email**

   chimchwee.wong@xmu.edu.my

3. **Microsoft Teams**

REFERENCES

1. **Problem Solving with C++ (10th Edition), Walter Savitch, Addison Wesley, 2018**
2. **Paul J. Dietel, C++ How to Program (10th Edition), Pearson, 2017.**
3. **Object Oriented Programming in C++, Fourth Edition, Robert Lafore.**

SOFTWARE REQUIRED

Software (C++ Compiler)

- **Codeblocks**



https://www.codeblocks.org/downloads/binaries/

- **Dev-C++**



https://sourceforge.net/projects/orwelldevcpp/

- **Other C++ compilers**

Visual Studio, Eclipse, Visual Studio Code, etc

SOFTWARE REQUIRED

Software (C++ Online Compiler)

- **Programiz online compiler**

  https://www.programiz.com/cpp-programming/online-compiler/

- **OnlineGDB**

  https://www.onlinegdb.com/online_c++_compiler#

COURSE LEARNING OUTCOMES

CLO No.	Description	Bloom's Taxonomy Level	PLO
1	Comprehend basic programming constructs of C and C++.	C2	1
2	Demonstrate an understanding of the concepts and principles of C and C++ programming by solving real-world problems in particular.	C3	2
3	Employ object-oriented concepts correctly in the codes	C3	2
4	Evaluate the given C/C++ code segments using algorithm tracing technique.	C4	7

COURSE INFORMATION

- This course is practical oriented.
- Lab Exercises and Assignments will be beneficial for you provided you try it on your own and understand them.
- Memorization will not help you to get a good result.
- Copying Code is
  - Strictly prohibited
  - Zero tolerance

INTRODUCTION TO C/C++

Where did C++ come from?

- **Derived from the C language**
- **C++ is an extension of the C language**
- **C++ is (almost) a superset of C.**
  - → **Most C programs can be compiled by C++ compiler.**
- **We may say C++ is created by adding object-oriented functionalities to C.**

C/C++ HISTORY

- C was developed by Dennis Ritchie at AT&T Bell Labs in the 1970s.
  - Used to maintain UNIX systems
  - Many commercial applications written in C
  - C is a procedure programming language.

- C++ developed by Bjarne Stroustrup at AT&T Bell Labs starting from 1979.
  - Overcame several shortcomings of C
  - C++ is an object oriented programming language.
  - C remains a subset of C++

STRUCTURE OF C PROGRAM

```
#include <stdio.h>

int main ( )
{
    printf (“Welcome to C” );
    return 0;
}
```

← Preprocessor Directives

← Main function header

Function body

STRUCTURE OF C++ PROGRAM

```
#include <iostream>

int main ( )
{
    std::cout << "Welcome to C" ;
    return 0;
}
```

← Preprocessor Directives

← Main function header

Function body

STRUCTURE OF C++ PROGRAM

```
# include <iostream>
using namespace std;

int main ( )
{
    cout << "Welcome to C" ;
    return 0;
}
```

← Preprocessor Directives

← Main function header

Function body

PREPROCESSOR DIRECTIVES

- The first 2 lines of the program

  #include <iostream> and **using namespace std**

  is called a **Preprocessor Directive**.

- Preprocessor Directive is the instruction that is given to the compiler to examine the code before actual compilation of code begins and resolves.

- It begins with the # sign and a keyword (reserve word) "include" followed by <name of header file>

HEADER FILES

- **Header files** are a part of the C / C++ Compiler and **contain definitions of standard library functions**.

- **For example;**

➢ **iostream**: The source file that contains declarations needed for input and output statements like cout/cin.

➢ **stdio.h**: The source file that contains declarations needed for input and output statements like print/input.

THE MAIN( ) FUNCTION

- Functions are the fundamental building blocks of C++. The first program consists of a single function called **main ( )**
- **The main() function:**
  - ❖ It indicates the beginning of a C++ program.
  - ❖ It must be included in every program.
  - ❖ When a C++ program is executed, the control goes directly to main () function.
  - ❖ The statements within the curly brackets (or braces) of this function are the main body of a program.
  - ❖ If the main function is not included, the program is not compiled and an error is generated.

SYNTAX OF MAIN FUNCTION

The syntax of the main() function :

```
int main ( )
 {
     body of main function
     (program statements) ;
 }
```

THE FUNCTION BODY

Braces: The braces (Curly brackets "{ }") indicate Begin and End of a function. Every function must use this pair of braces.

Program Statements: The basic unit of a C++ program. In our program we've only one program statement i.e.

cout << "Welcome to C++" ;

- This statement tells the compiler to display the quoted phrase.
- A semicolon (;) indicates the end of the statement.
- Each statement has to be followed by a semicolon otherwise, a compilation error will occur.

OUTPUT USING COUT

- The identifier **cout** (pronounced as " C out") is a predefined object in C++, called as Standard Output Stream. It follows data to be displayed.

- The operator **<<** is called the **Insertion** or **Put to** operator.

- In our example, the phrase "Welcome to C++" is an example of a **String Constant** or **String Literal**, is directed to **cout** by the insertion operator **<<** and is to be displayed on the output.

THE ENDL MANIPULATOR

- **Manipulators** are instructions to the output stream that modify the output in various ways.

- This causes a linefeed to be inserted into the stream, so that the subsequent text is displayed on the next line. It has the same effect as the **'\n'** character.

```
cout << "1st Line text goes here" << endl;

cout << "2nd line text..." << endl;

cout << "3rd line goes here\n 4th line text\n";
```

COMMENTS

- A comment statement is a non-executable statement. It is used to add remarks in a program. The compiler ignores the comments given in the program. Comments are used to explain, what a program is doing. Comments can be written anywhere in a program. Syntax:

```
// comments                for single-line comments
/* comments……              for multi-line comments
…….. */
```

ESCAPE SEQUENCES

- Escape sequences are written as a backslash character (\) followed by one or more control characters and are used to control the way output is displayed.

- There are many escape sequences in C++. The newline escape sequence (\n) is just one of them.

COMMON ESCAPE SEQUENCES

Escape Sequence	Name	Description
\n	Newline	Causes the cursor to go to the next line for subsequent printing.
\t	Horizontal tab	Causes the cursor to skip over to the next tab stop.
\a	Alarm	Causes the computer to beep.
\b	Backspace	Causes the cursor to back up, or move left one position.
\r	Return	Causes the cursor to go to the beginning of the current line, not the next line.
\\	Backslash	Causes a backslash to be printed.
\'	Single quote	Causes a single quotation mark to be printed.
\"	Double quote	Causes a double quotation mark to be printed.

A PROGRAM TO DEMONSTRATE ESCAPE SEQUENCES & COMMENTS

```
# include<iostream>
using namespace std;
int main()  //beginning of main function
{
  cout << "\n \"XIAMEN \t UNIVERSITY\" ";   //printing outputs
  cout << "\n \'MALAYSIA\' ";
  return 0;
}
```

How the Output looks like ???

4. TESTING AND DEBUGGING

- **Bug**
  - **A mistake in a program**

- **Debugging**
  - **Eliminating mistakes in programs**

- **Can be done manually or using a debugger utility**

MAIN TYPES OF ERRORS

- **Syntax errors**
  - Violation of the grammar rules of the language
  - Discovered by the compiler
  - Error messages may not always show correct location of errors
- **Run-time errors**
  - Errors detected by the computer at run-time
- **Logic errors**
  - Errors in the program's algorithm
  - Most difficult to diagnose
  - Computer/compiler does not recognize such errors

SYNTAX ERRORS VS LOGIC ERRORS

- **A syntax error occurs when the program does not follow the standard rule of the programming language. The compiler detects the syntax errors.**
- **Examples of syntax errors:**

```
cout << " The sum is  << num1+num2;
return 0
```

- **A logic error occurs when the program does not run as the programmer expect.**
- **The programmer must detect logic errors by himself. This process is called debugging.**

SYNTAX ERRORS VS LOGIC ERRORS

- **Examples of logical errors:**
  - **cout << " The sum of 3 + 4 = “ << 10;**
  - **a = 10;**
    b = 5;
    product = a / b;
    cout << “The product of a and b = “ << product;

DISPLAY 1.8 A Sample C++ Program

```
1    #include <iostream>
2    using namespace std;

3    int main( )
4    {
5        int number_of_pods, peas_per_pod, total_peas;

6        cout << "Press return after entering a number.\n";
7        cout << "Enter the number of pods:\n";

8        cin >> number_of_pods; 10

9        cout << "Enter the number of peas in a pod:\n";
10       cin >> peas_per_pod; 9
11       total_peas = number_of_pods * peas_per_pod;
12       cout << "If you have ";
13       cout << number_of_pods;
14       cout << " pea pods\n";
15       cout << "and ";
16       cout << peas_per_pod;
17       cout << " peas in each pod, then\n";
18       cout << "you have ";
19       cout << total_peas;
20       cout << " peas in all the pods.\n";

21       return 0;

22   }
```

How are the output looks like if the 1st input is 10 and 2nd input is 9?

ANSWER

```
Press return after entering a number.
Enter the number of pods:
10
Enter the number of peas in a pod:
9
If you have 10 pea pods
and 9 peas in each pod, then
you have 90 peas in all the pods.
```

DISPLAY 1.9 Layout of a Simple C++ Program

```
1     #include <iostream>
2     using namespace std;
3
4     int main()
5     {
6         Variable_Declarations
7
8         Statement_1
9         Statement_2
10        ...
11        Statement_Last
12
13        return 0;
14    }
```

EXTRA READINGS

HARDWARE & SOFTWARE

- The actual physical components that make up a computer installation are referred to as hardware.
- A set of instructions for a computer to follow is called a program.
- The collection of programs used by a computer is referred to as the software for that computer.

COMPUTERS

Three main classes of computers:

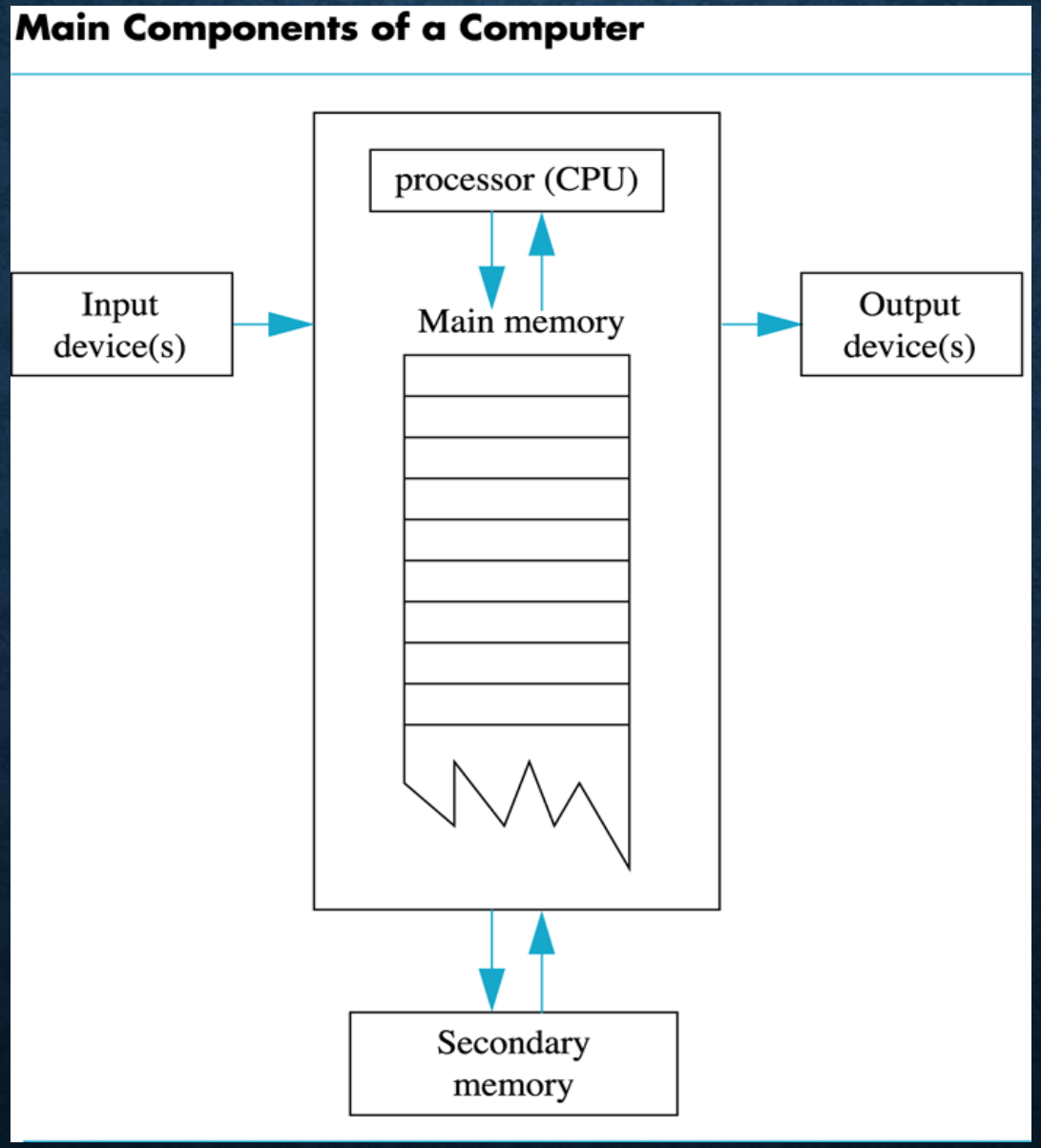
- **PCs (Personal Computer) / Laptop**
  - Relatively small used by one person at a time
- **Workstation**
  - Larger and more powerful computer than a PC
- **Mainframe**
  - Larger than the above 2 classes of computers
  - Requires support staff
  - Shared by multiple users

COMPUTER ORGANIZATION

Five main components

1. **Input devices**
   - Allows user to communicate to the computer
   - Devices that are used to enter data to the computer for performing tasks.
2. **Output devices**
   - Allows computer to communicate to the user
3. **Processor (CPU)**
   - Interprets and executes the commands from the computer's other hardware and software.
4. **Main memory**
   - Temporary memory locations containing the running program
5. **Secondary memory**
   - Permanent memory for storing data

COMPUTER ORGANIZATION CONT…



COMPUTER MEMORY

- **Main Memory**
  - **Long list of memory locations**
    - Each contains zeros and ones
    - Can change during program execution
  - **Binary digit or Bit**
    - A digit that can only be **zero or one**
  - **Byte**
    - Each memory location has **eight bits**
  - **Address**
    - Number that identifies a **memory location**

Character	Binary Code	Character	Binary Code	Character	Binary Code
A	01000001	a	01100001	!	00100001
B	01000010	b	01100010	"	00100010
C	01000011	c	01100011	#	00100011
D	01000100	d	01100100	$	00100100
E	01000101	e	01100101	%	00100101
F	01000110	f	01100110	&	00100110
G	01000111	g	01100111	'	00100111
H	01001000	h	01101000	(	00101000
I	01001001	i	01101001	)	00101001
J	01001010	j	01101010	*	00101010
K	01001011	k	01101011	+	00101011
L	01001100	l	01101100	,	00101100
M	01001101	m	01101101	-	00101101
N	01001110	n	01101110	.	00101110
O	01001111	o	01101111	/	00101111
P	01010000	p	01110000	0	00110000
Q	01010001	q	01110001	1	00110001
R	01010010	r	01110010	2	00110010
S	01010011	s	01110011	3	00110011
T	01010100	t	01110100	4	00110100
U	01010101	u	01110101	5	00110101
V	01010110	v	01110110	6	00110110
W	01010111	w	01110111	7	00110111
X	01011000	x	01111000	8	00111000
Y	01011001	y	01111001	9	00111001
Z	01011010	z	01111010	?	00111111
				@	01000000

All characters and digits are represented in a byte.

COMPUTER MEMORY CONT…

Main memory

- stores instructions and data while a program is running
- **Example: RAM (Random Access Memory)**

Secondary memory

- Stores instructions and data between sessions
- A file stores data or instructions in secondary memory
- **Example: Pen-drive, HDD, SSD, magnetic tape.**

THE PROCESSOR

- Commonly also called the **CPU (Central Processing Unit)**
- **It interprets and executes program instructions**
- **Typical capabilities of CPU include:**
  - **add**
  - **Subtract**
  - **multiply**
  - **Divide**
  - **move data from location to location**



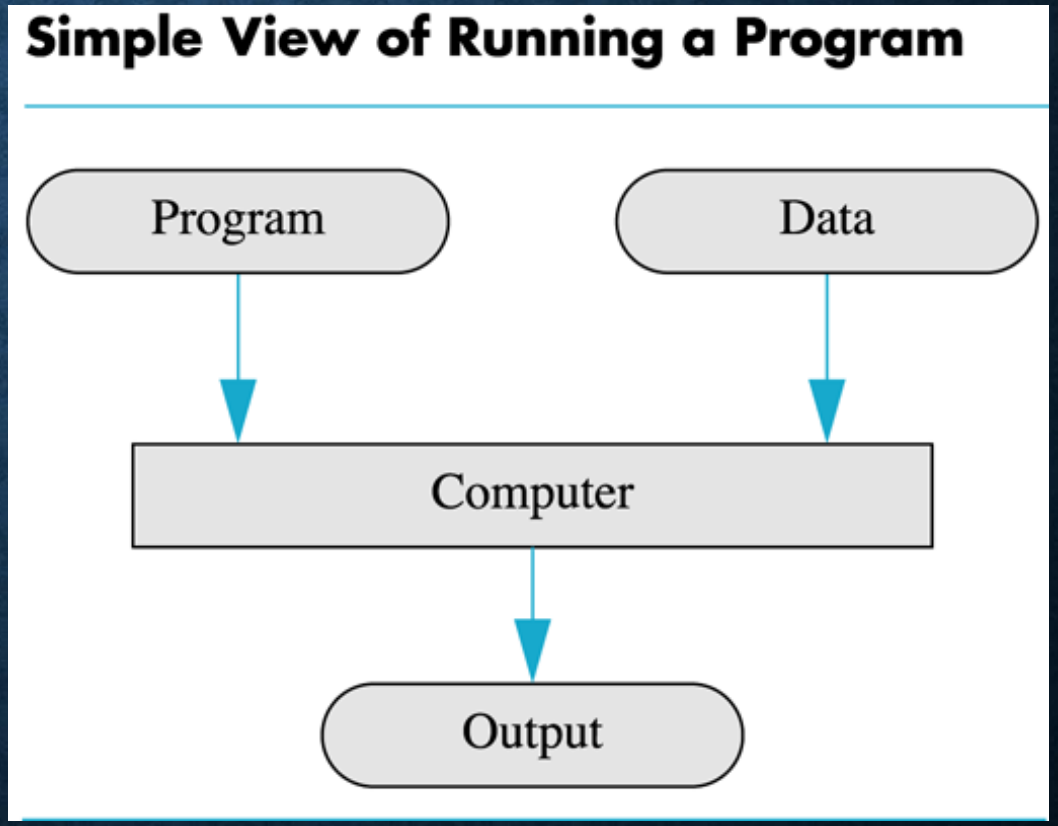
COMPUTER SOFTWARE

- The **Operating System (OS)**
  - **It is a computer program**
  - **Allows us to communicate with the computer**
  - **Allocates computer resources**
  - **Responds to user requests to run other programs**

- Common operating systems include…
  - **UNIX, Linux, DOS, Windows, Mac OS**

COMPUTER INPUT

- Computer input generally consists of two parts
  - A program
  - Some data



LOW LEVEL PROGRAMMING LANGUAGES

- An **assembly language** command such as

  ADD X Y Z

  might mean add the values found at x and y in memory, and store the result in location z

- Assembly language must be translated to **machine language** (zeros and ones)

  0110 1001 1010 1011

- **The CPU can only execute machine language**

Machine Language

```
<main+0>: 01010101 10001001 11100101 10000011 11100100 11110000 10000011 11101100
<main+8>: 00100000 11101000 00011010 01000000 00000000 00000000 11000111 01000100
<main+16>:00100100 00000100 00100100 01110000 01000111 00000000 11000111 00000100
<main+24>:00100100 10000000 01011111 01000111 00000000 11101000 10100110 10001100
<main+32>:00000110 00000000 10001101 01000100 00100100 00010100 10001001 01000100
```

Assembly Language

```
<main>:          push   %ebp
<main+1>:        mov    %esp,%ebp
<main+3>:        and    $0xfffffff0,%esp
<main+6>:        sub    $0x20,%esp
<main+9>:        call   0x40530c <__main>
<main+14>:       movl   $0x477024,0x4(%esp)
<main+22>:       movl   $0x475f80,(%esp)
<main+29>:       call   0x469fac <operator<<>
<main+34>:       lea    0x14(%esp),%eax
<main+38>:       mov    %eax,0x4(%esp)
```

HIGH LEVEL PROGRAMMING LANGUAGES

- **Common programming languages include …**

  C C++ Java Pascal Visual Basic FORTRAN

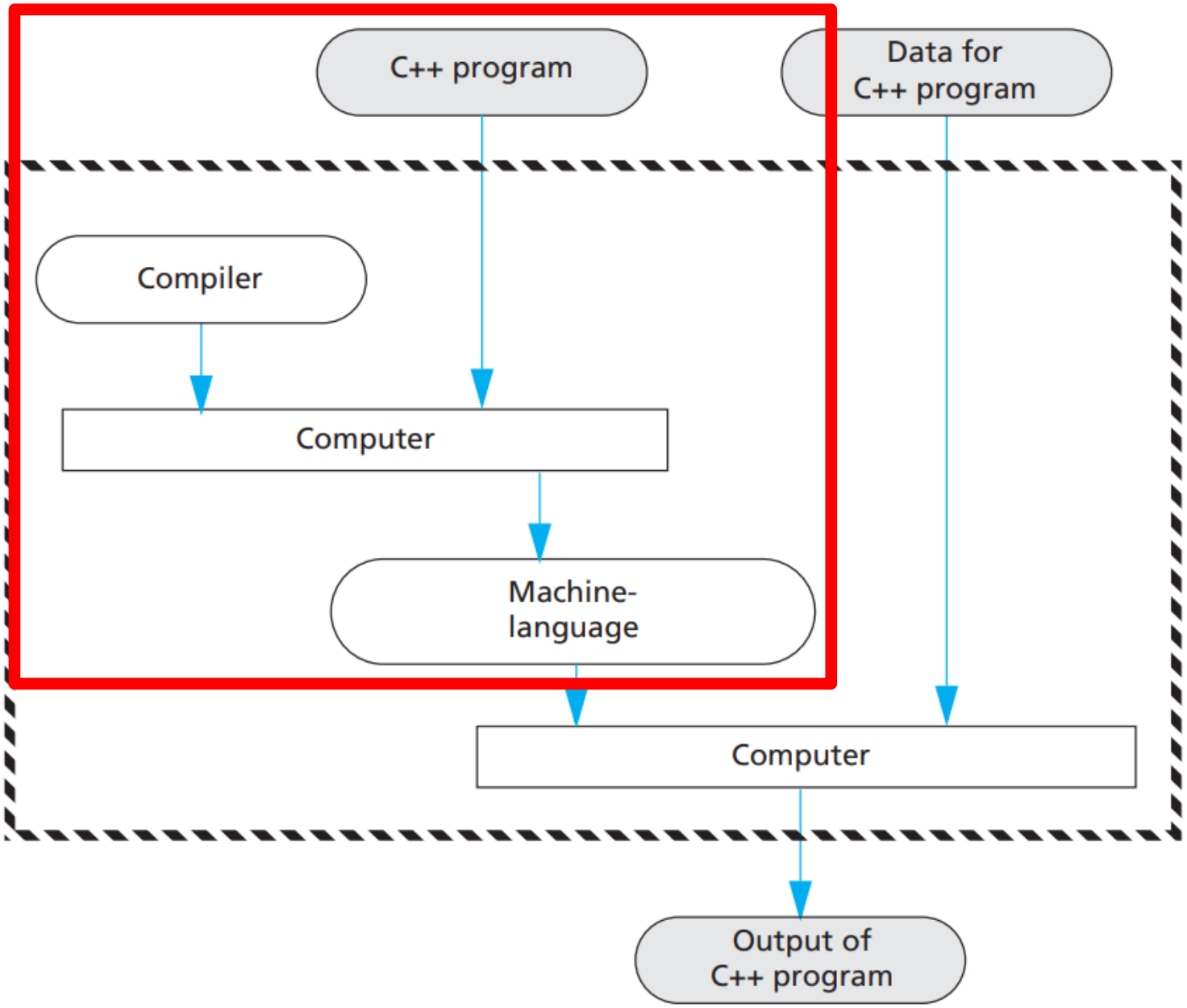
  COBOL Lisp Scheme Ada

- **These high–level languages**
  - **Resemble human languages**
  - **Are designed to be *easy* to read and write**
  - **Use more complicated instructions than the CPU can follow**
  - **Must be translated to 0's and 1's for the CPU to execute a program**

COMPILERS

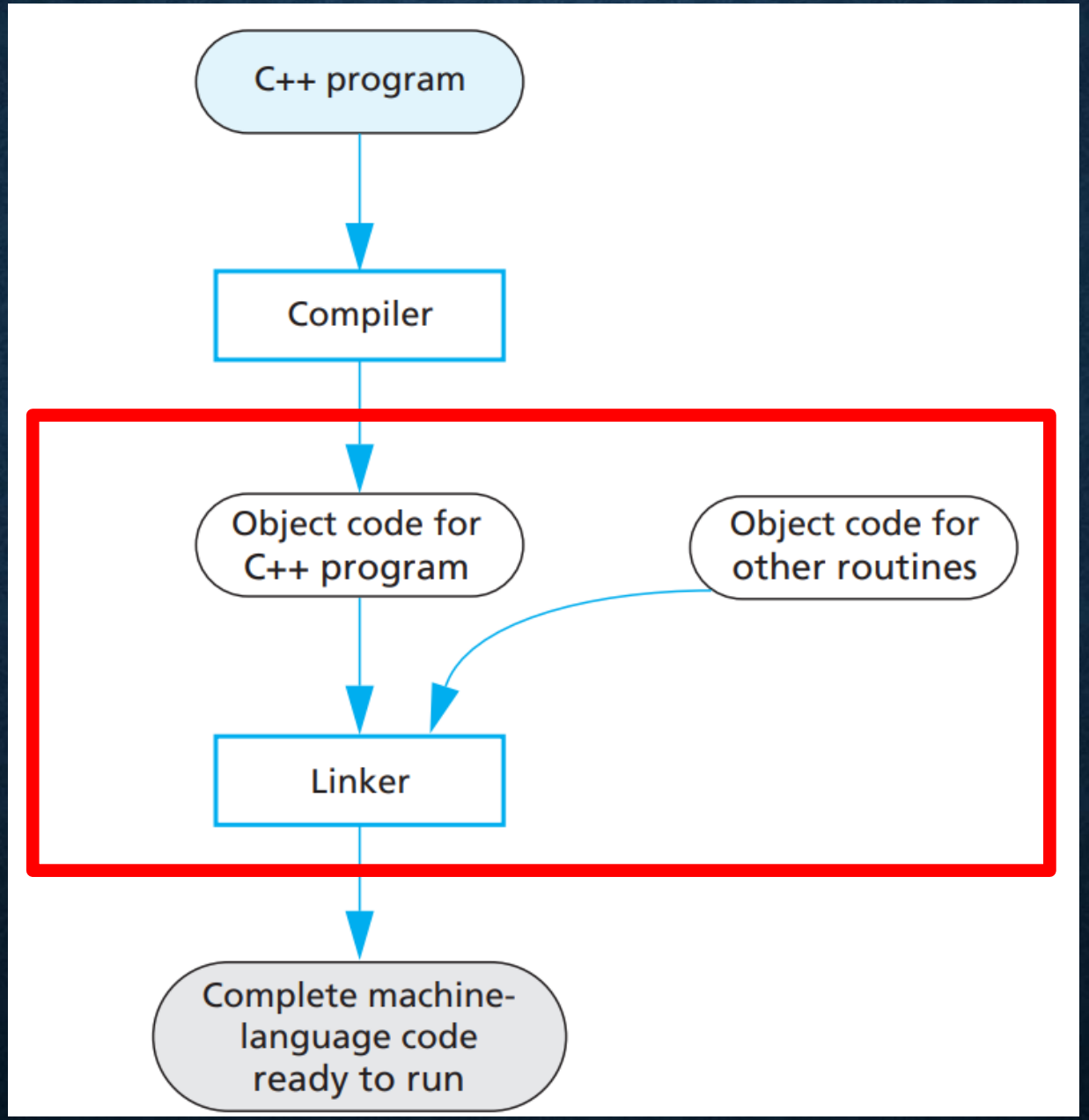
- **Compiler is a program that translates high-level language to machine language**
  - **Source code**
    - the original program in a high level language
  - **Object code**
    - the translated program in machine language

DISPLAY 1.4 Compiling and Running a C++ Program (Basic Outline)



LINKERS

- The linker combines all external programs (such as libraries and other shared components) with the machine code to create a final executable program.


C++ program
Compiler
Object code for C++ program
Object code for other routines
Linker
Complete machine-language code ready to run

WHAT IS PROGRAMMING?

- A **computer program** is…
  - A set of instructions for a computer to follow
  - Also known as software
  - A structured combination of data and instructions used to operate a computer to produce a specific result.

- Programmers use Programming languages to write codes line by line and construct computer programs.

- Running a program means executing the instructions line by line and produce a expected result.

WHAT IS COMPUTER SOFTWARE?

- **Computer software is the collection of programs used by a computer.**

- **Computer software includes:**
  - **Editors**
  - **Translators**
  - **System software**
  - **Application software, etc.**